

第 12 屆成功大學高中生暑期程式設計邀請賽 - 初賽

比賽規則

1. 本次競賽試題共 9 題，每題皆採 ICPC 制評分。
2. 本次競賽使用系統：DOMJudge。
3. 程式碼均需上傳至系統進行評測，並得到「Correct」的結果才算答對。
4. 如果該題未通過，除了「Compiler-Error」之外的任何結果均有罰時 20 分鐘。
5. 除了特殊評測的題目之外，其餘題目請依照輸出格式要求輸出，避免系統判定為 Wrong Answer。
6. 如果對競賽「試題」有任何問題，請善用平台的提問系統進行提問。
7. 如果系統在競賽中出現任何「設備」上的問題，請立即舉手向裁判提出。
8. 請勿在競賽中做出會干擾競賽進行的行為。
9. 題目順序並沒有按照難度順序編排，建議把所有題目都看過一次。
10. Good Luck & Have Fun!



NCKU
PROGRAM DESIGN COMPETITION



成功大學資訊工程學系暨研究所

Department of Computer Science and Information Engineering

題目名稱	時間限制	記憶體限制
Amazing tree combination - 神奇樹木	1.0 秒	256 MB
Boom	3.0 秒	2048 MB
Class Rush in Midsummer - 盛夏的跑堂大作戰	1.0 秒	256 MB
Deadline	2.0 秒	256 MB
Energy - 能量擴張	0.5 秒	256 MB
Flow - 網路流	1.0 秒	256 MB
Game design - 關卡設計	1.0 秒	256 MB
Hamburger	1.0 秒	256 MB
I like immediate apologies	1.0 秒	256 MB

A. Amazing tree combination - 神奇樹木

問題敘述

你重生了，意外穿越到了名為 Latent 的古老王國，並成為了一名受人敬重的皇家伐木師。剛重生的這天，你接到了一項艱鉅的任務：城門口有一排共 n 棵並排成一列的樹木，每一棵樹都有各自的高度 h_i 。

相傳，Latent 的開國君王在臨終前留下了一道遺囑，他認為唯有呈現「起伏律動」的森林，才能守護國家的氣運。這種符合神聖律動的樹林被稱為「神奇樹木組合」，其規則十分嚴苛：

經過修整後剩下的 m 棵樹木，從左而右的第一棵樹開始，樹木的高度必須像山脈的脊線一樣，在「巔峰」與「深谷」之間不斷交替。具體來說，對於滿足 $1 < i < m$ 的 i ，如果第 i 棵樹比第 $i-1$ 棵高，那第 $i+1$ 棵就必須比第 i 棵矮；反之亦然，如果第 i 棵樹比第 $i-1$ 棵低，那第 $i+1$ 棵就必須比第 i 棵高。因此一個長度為 m 的「神奇樹木組合」以數學方式表示，會滿足以下其中一種情況：

- $h[i-1] > h[i] < h[i+1]$ ($\forall 1 < i < m, i$ 為奇數) 且 $h[i-1] < h[i] > h[i+1]$ ($\forall 1 < i < m, i$ 為偶數)。
- $h[i-1] < h[i] > h[i+1]$ ($\forall 1 < i < m, i$ 為奇數) 且 $h[i-1] > h[i] < h[i+1]$ ($\forall 1 < i < m, i$ 為偶數)。

時光荏苒，這片原本充滿靈氣的森林因為過度生長而失去了規律。身為擁有現代演算法智慧的穿越者，你的職責是決定要砍伐哪些樹木來重塑護國大陣。由於這些樹木非常神奇，一旦被砍伐，便會化作點點靈光徹底消失不見，不會留下任何殘枝斷木；隨後，剩下的樹木會保持原本的相對順序，自動向中間靠攏，重新排成一列。

這是一項關乎 Latent 國運的浩大工程。當代國王想知道，在所有可能使得剩下的樹木數量非零的砍伐方案中，究竟有多少種不同的組合，能讓剩下的樹木重新恢復那神聖的「起伏律動」？由於答案可能非常大，國王要求你將最終計算出的方案總數對 $10^9 + 7$ 取模。

輸入說明

第一行包含一個整數 n ，代表城門口有 n 棵樹木。

第二行包含 n 個以空格分隔的整數 $h_1, h_2, \dots, h_n$ ，代表把樹木排成一行後，從左到右每棵樹的初始高度。

輸出說明

輸出一個整數，代表所有可能的砍伐方案中，能組成「神奇樹木組合」的方法數量。

測資限制

- $1 \leq n \leq 2 \times 10^5$
- $1 \leq h_i \leq 10^9$

範例測資

範例輸入 1

5
1 3 2 5 4

範例輸入 2

7
2 4 1 2 3 3 4

範例輸入 3

3
1 2 2

範例輸出 1

25

範例輸出 2

51

範例輸出 3

5

B. Boom

問題敘述

特務黃昏和安妮亞正在進行一場遊戲，特務黃昏的目標是從迷宮的角落移動到終點線之後。迷宮可以視為一張 $n \times n$ 的二維平面上的方格圖，黃昏從左下角 $(0,0)$ 進入迷宮，每次可以移動到與當前方格有公共邊的方格，直到越過終點線 $y = n$ ，而安妮亞要阻止特務抵達終點線，阻止的方式是透過在迷宮中佈置 $n - 1$ 個「要你命 3000」——某種安全無害的傳送陷阱，當黃昏進入一個佈置了「要你命 3000」的方格，就會爆炸然後被傳送回 $(0,0)$ ，唯一的缺點是「要你命 3000」彼此之間不能佈置在同一直排或橫排，也不能佈置在任何 $y = 0$ 的方格上。

身負維繫東西國和平的重責大任，工作結束後的特務黃昏，或許是因為過於疲憊，不幸追撞了黑色的高級車，沒時間思考策略。於是他希望你幫他寫一個程式，告訴他使用最佳策略的情況下，最差可能踩中幾次陷阱。然而黃昏並不知道安妮亞有讀心的特殊能力，安妮亞可能會在不違反佈置規則的情況下，於黃昏移動後重新安排未被觸發過的陷阱位置。

輸入說明

第一行有一個正整數 n ，代表迷宮的大小。

接下來每次你輸出移動指令之後，會收到三種可能的系統回復：

SAFE! / BOOM! / GOAL!，請依據收到的訊息執行下列動作：

SAFE!: 上次移動指令之後，移動到的新格子沒有陷阱，你需要輸出下個移動指令。

BOOM!: 上次移動指令之後，移動到的新格子有陷阱，黃昏的座標現在變成 $(0,0)$ ，你需要輸出下個移動指令。如果你接受到第 $op + 1$ 次 **BOOM!**，表示你的策略失敗了，評測程式會結束，結果為 WA。

GOAL!: 上次移動指令之後，黃昏越過了終點線，請在接受到 **GOAL!** 之後結束你的程式，評測程式會結束，如果你輸出的 op 是正確的，也就是不可能有任何策略能保證觸發小於 op 次陷阱並越過終點，則結果為 AC；否則結果為 WA。不要輸出任何其他指令，否則可能會得到無法預料的結果。

如果你的移動指令會使你離開迷宮，並且沒有越過終點線，則評測程式會結束，結果為 WA。

輸出說明

讀入 n 之後，你需要輸出一個正整數 op ，且滿足 $op < n$ ，表示你覺得至少會踩中多少次陷阱。

之後你需要輸出 **UP/DOWN/RIGHT/LEFT** 其中之一，指示黃昏應該如何移動。

每次你輸出任何指令之後，都需要清空緩衝區，下列是一些清空緩衝區的方法：

- `fflush(stdout)` in C/C++;
- `System.out.flush()` in Java;
- `stdout.flush()` in Python;

測資限制

- $2 \leq n \leq 10^3$

範例測資

範例輸入 1

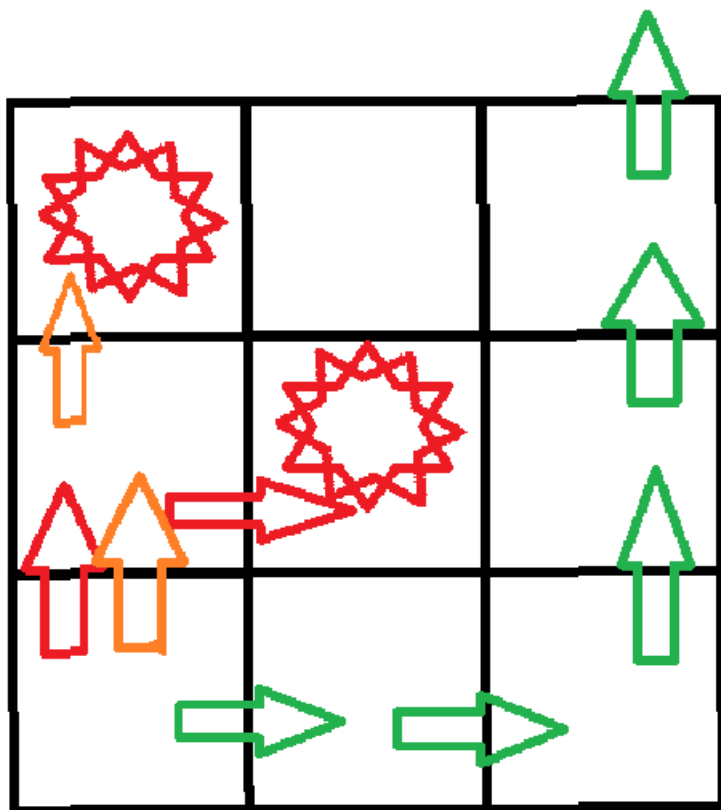
3
SAFE!
BOOM!
SAFE!
BOOM!
SAFE!
SAFE!
SAFE!
SAFE!
GOAL!

範例輸出 1

2
UP
RIGHT
UP
UP
RIGHT
RIGHT
UP
UP
UP

下一頁有範例解釋。

範例測資可以參考下圖：



圖中紅色箭頭為黃昏第一次的行進路線，橘色是第二次，綠色是第三次，齒輪圖標表示陷阱位置，陷阱一旦觸發表示開陷阱在接下來的遊戲中都不能被移動，並且同直排與橫排都不可能觸發陷阱。

範例測資中宣言的 op 為 2，經過一系列移動後成功達到終點，觸發陷阱的次數沒有超過 op ，並且不可能有任何方法能保證觸發 2 次以下陷阱並越過終點，因為安妮亞可以在你第一次抵達 $y = 1$ 的格子時移動一個陷阱到你的腳下，第一次抵達 $y = 2$ 的格子時移動再一個陷阱到你的腳下，無論黃昏如何移動，必定觸發至少兩次陷阱，因此 $op = 2$ 是正確的，最後評測結果為 AC。

C. Class Rush in Midsummer - 盛夏的跑堂大作戰

問題敘述

炎炎夏日，你在學校的「綜合大樓」剛上完微積分，接下來只有 10 分鐘的時間，必須趕到校園另一頭的「資訊大樓」上課。

校園的地圖可以看作是一個 $N \times M$ 的網格。行 (Column) 從 1 到 M 代表從綜合大樓到資訊大樓的 M 個區段；列 (Row) 從 1 到 N 則代表校園中平行的 N 條走廊。每個網格點 (i, j) 都有一個「陰涼度」 $V_{i,j}$ ，數值越高代表該路段越涼爽。

為了避免遲到，你的移動規則如下：

1. 你必須從第 1 行（最靠近綜合大樓）的任意一列出發，並抵達第 M 行（最靠近資訊大樓）的任意一列。
2. 每次移動必定是往右前進一行。也就是說，如果你現在位於 (R_j, j) ，下一步必須走到 $(R_{j+1}, j+1)$ 。
3. 為了避免過度繞路，相鄰兩步的落差不能超過 D 。亦即： $|R_{j+1} - R_j| \leq D$ 。

這趟跑堂之旅的「痛苦瓶頸」取決於你所經過的 M 個格子中，陰涼度的最小值。

幸運的是，你出門前準備了「急凍防曬噴霧」，在這趟路途你最多可以使用 K 次噴霧（每個格子最多使用一次）。當你在某一個網格使用噴霧時，你可以完全無視該地的酷熱，該格的陰涼度將被暫時視為 ∞ 。

請幫自己規劃出一條合法的路線，並妥善分配噴霧，使得整趟路線在未防護狀態下的「痛苦瓶頸」越高越好。請輸出這個最大可能的痛苦瓶頸。

輸入說明

第一行包含四個整數 N, M, K, D ，相鄰數字以空白隔開。

接下來有 N 行，每行包含 M 個整數。其中第 i 行的第 j 個整數代表網格的陰涼度 $V_{i,j}$ 。

測資限制

- $1 \leq N, M \leq 1000$
- $0 \leq K < M$
- $0 \leq D \leq N$
- $1 \leq V_{i,j} \leq 10^9$

輸出說明

請輸出一個整數，代表路線中可能達成的最大「痛苦瓶頸」。

範例測資

範例輸入 1

```
4 5 1 1
10 20 30 10 50
50 10 10 40 10
20 50 20 10 30
10 10 50 20 10
```

範例輸出 1

40

【範例 1 說明】

最佳路線為 $(2, 1) \rightarrow (3, 2) \rightarrow (2, 3) \rightarrow (2, 4) \rightarrow (1, 5)$ 。

經過的陰涼度依序為 50, 50, 10, 40, 50。垂直落差依序為 1, 1, 0, 1，均沒有超過 $D = 1$ 。

此時我們對第三步 $(2, 3)$ 使用唯一的一次防曬噴霧，使其陰涼度變為 ∞ 。

路線剩餘的陰涼度為 50, 50, ∞ , 40, 50，其中的最小值為 40。可以證明沒有其他路線與噴霧組合能得到大於 40 的結果。

範例輸入 2

```
5 6 2 2
15 20 10 25 15 20
25 50 20 15 20 10
30 10 15 50 10 50
10 25 50 10 30 15
20 15 10 25 20 10
```

範例輸出 2

50

【範例 2 說明】

(此範例對應的 $N = 5, M = 6, K = 2, D = 2$ ，其中起點與終點附近各有一個陰涼度僅有 30 的必經路段)

最佳路線為 $(3, 1) \rightarrow (2, 2) \rightarrow (4, 3) \rightarrow (3, 4) \rightarrow (4, 5) \rightarrow (3, 6)$ 。

經過的陰涼度依序為 30, 50, 50, 50, 30, 50。相鄰兩步行數最大落差為 $|4 - 2| = 2$ ，符合 $D = 2$ 的限制。

此時我們在第一步 $(3, 1)$ 與第五步 $(4, 5)$ 分別使用防曬噴霧，將兩處的陰涼度變為 ∞ 。

路線剩餘的陰涼度為 $\infty, 50, 50, 50, \infty, 50$ ，其中的最小值為 50。

D. Deadline

問題敘述

有 n 個任務要在一顆 CPU 上面跑，他們可以以任意順序被執行，一次只會有一個任務被執行，執行一個任務需要多少久可以由它需要的 CPU cycle 數字來評價。一個 CPU cycle 是多少時間取決於 CPU 頻率，也就是單位時間中有多少個 CPU cycle。每個任務有 dl_i 和 c_i 兩個數字。 dl_i 代表在起始時間後 dl_i 微秒前這個任務需要被執行完（剛好在 dl_i 微秒時被執行完被定義為成功完成）， c_i 代表這個任務需要耗 c_i 個 CPU cycle 來完成。一微秒有多少個 CPU cycle 是由 CPU 的頻率決定的，CPU 頻率是 f 代表每微秒有 f 個 CPU cycle。很顯然的， f 越大代表著單位時間有更多的 CPU cycle，也代表完成一個任務所需的時間越少。不過，CPU 的頻率 f 是不能無限制的增加的，你的任務是找到一個最低的整數頻率 f 使得存在一種任務編排的方式讓所有任務都成功在期限內完成。另外注意，不會有任何 dl_i 或 c_i 是 0。

以第一筆範測為例，有 3 個任務，編號 0、1、2，如果說 $f = 2$ ，代表每微秒有 2 個 CPU cycle，那一個 CPU cycle 就是 0.5 微秒。如果以 1、0、2 這樣的順序去執行的話，執行完編號 1 的任務後，時間是 $0.5 \times 2 = 1$ 微秒，在 $dl_1 = 1$ 內，執行完編號 0 的任務後，時間是 $1 + 0.5 \times 4 = 3$ 微秒，在 $dl_0 = 3$ 內，執行完編號 2 的任務後，時間是 $3 + 0.5 \times 2 = 4$ 微秒，在 $dl_2 = 5$ 內。所以這是個能成功執行完所有任務的頻率。

輸入說明

第一行有個正整數 n 代表著有多少任務。

接下來有 n 行，每行有 2 個正整數， dl_i 和 c_i ，分別代表第 i 個任務的時限與所需要耗的 CPU cycle 數。

輸出說明

輸出一個正整數 f ，代表 CPU 頻率至少要多少才能讓所有任務在適當排序下完成。

測資限制

- $1 \leq n \leq 5 \cdot 10^5$
- $1 \leq dl_i, c_i \leq 10^9$

範例測資

範例輸入 1

3
3 4
1 2
5 2

範例輸入 2

2
1 5
3 5

範例輸出 1

2

範例輸出 2

5

E. Energy - 能量擴張

問題敘述

在一個實驗室中，科學家正在進行能量融合實驗。

實驗室中共有 n 個能量源，每個能量源 i 包含 x_i 個不同能階的粒子，第 i 個能量源中的第 j 個粒子具有能量 e_{ij} ，除了能量源，實驗室還有 z 個能量放大器，第 i 個能量放大器的倍率為 t_i

為了啟動核心，科學家必須從每一個能量源中各挑選出一個粒子，並挑選出一個能量放大器來使用。如果選出的 n 個粒子能量分別為 $e_{1,j_1}, e_{2,j_2}, \dots, e_{n,j_n}$ ，且選擇第 g 個能量放大器，則該組合的「總能量」計算方式如下：

$$\text{Total Energy} = \left(\sum_{i=1}^n e_{i,j_i} \right) \times t_g$$

兩種選法被視為不同，只要它們選擇的粒子組合 $(j_1, j_2, \dots, j_n)$ 不同，或是選擇的能量放大器編號不同。

你的任務是計算在所有可能的組合中，總能量第 k 小的選法之總能量為多少

輸入說明

第一行有兩個整數 n, k ，代表能量源數量和目標排名。

接下來有 n 行，第 i 行的第一個整數 x_i ，接著有 x_i 個整數 e_{ij} ，代表能量源的數量和能量

最後一行的第一個整數代表 z ，接著有 z 個整數代表 t_i ，代表能量放大器的數量和倍率

- $1 \leq n \leq 10^5$
- $1 \leq x_i, z \leq 10$
- $1 \leq k \leq \min(10^5, (\prod_{i=1}^n x_i) * z)$
- $1 \leq e_{ij} \leq 10^9$
- $1 \leq t_i \leq 100$

輸出說明

輸出一個整數代表第 k 小總能量

範例測資

範例輸入 1

```
2 4
3 4 2 6
2 1 3
2 1 2
```

範例輸出 1

6

範例輸入 2

```
3 10
4 1 5 3 10
3 2 3 3
5 1 3 4 6 6
2 1 2
```

範例輸出 2

7

F. Flow 網路流

問題敘述

繼去年 pudding164253 在邀請賽出了去海邊的題目，又到了接近夏天的季節。

pudding164253 跟和沙老師在討論今年夏天該做甚麼，便想起以前去隅田川的事，

雖然當年晴空塔還叫天空樹，而且花火大會主要就在河的兩側而已，

但為了作為未來其他作品的素材，pudding164253 決定去平行世界的花火大會探勘。

在花火大會中有一堆道路，道路的交匯點是路口，

而任意兩個直接相鄰連通的路口中間相連的道路便稱為一段路段，

每一個路段上都有一間攤販，

但是 pudding164253 想像力太差了，他想像出來的攤販只有少數幾種，

和沙老師與 pudding164253 一人站在一個路口，pudding164253 要走到和沙老師那，

並且因為他太貪吃了，所以他會買下經過的所有路段上的攤販的食物，

可是走過太多路段會讓和沙老師等太久，這樣不好，

連續兩次買的攤販種類不同也不好，因為要把一樣的東西放在一起，

pudding164253 就得停下來花時間整理手上的東西，

而買太多種食物也不好，這樣要吃很久，

但是 pudding164253 太笨了，他只看到道路們形成了一種網路的長相，

卻沒有寫過網路流 24 題，所以只好請你幫他找到一條耗時最短的路徑。

一條路徑的耗時定義為路徑中包含了幾個路段，

加上任兩個相鄰路段的攤販不同的數量，

再加上整條路徑上經過多少種相異的攤販，

由於和沙老師不在乎 pudding164253 怎麼走的，只需要告訴他花了多久就好。

保證存在至少一條路徑使 a 路口和 b 路口連通。

(另外，pudding164253 跟和沙老師上次去已經是 2010 還是 2011 的事了，)

(所以這份題敘在未來某天又去一次之後會復刻。)

輸入說明

第一行 $n\ m\ k\ a\ b$ 五個空格隔開的數字，

代表花火大會總共有 n 個路口、 m 個路段、 k 種攤販，

pudding164253 在 a 路口，要走到和沙老師所在的 b 路口，

接下來的 m 行，每行有 u, v, x 三個數字以空格隔開，

每行代表一條路段連接兩個路口 u, v ，且路段上的攤販種類是 x ，

輸出說明

共輸出一個數字，代表從 a 路口走到 b 路口的最短耗時是多少。

測資限制

1. 保證 a, b 連通
2. m 行中不存在任意兩個路段連接相同的兩個路口，
且每一個路段所連接的兩端不會是同一個路口。
3. $1 \leq u, v, a, b \leq n \leq 16425$
4. $1 \leq m \leq 16425$
5. $1 \leq x \leq k \leq 9$

範例測資

範例輸入 1

```
4 5 2 1 4
1 2 1
1 3 2
2 3 1
2 4 2
3 4 1
```

範例輸出 1

```
4
```

【範例 1 說明】

走 1->2->3->4 共 3 個路段，0 個相鄰路段的攤販是相異的，

整條路徑上攤販共有 1 種，故總耗時為 $3+0+1=4$ 。

同理，走 1->2->4 或 1->3->4 都是共 2 個路段，

1 個相鄰路段的攤販是相異的，整條路徑上攤販共有 2 種，

故總耗時為 $2+1+2=5$ 。

除此之外的路徑明顯都不會有更短的耗時，故答案為 4。

G. Game design - 關卡設計

問題敘述

IZhna 正在製作一個冒險闖關遊戲。在這款遊戲中，玩家需要打怪、通關並獲取獎勵。IZhna 總共設計了 n 個關卡。每個關卡都有一個難度值，該值只能是 -1 、 0 或 1 。為了確保遊戲充滿樂趣 — 既不會難到卡關，也不會無聊到讓人睡著 — 難度的設定上有一個嚴格的規定：

令 c_i 為第 i 個關卡的最終難度。對於每個關卡 i ($1 \leq i \leq n$)，從第 1 關到第 i 關的難度總和必須恰好落在 0 到 1 之間，即前綴和 s_i 必須滿足 $0 \leq s_i \leq 1$ 。

因為關卡數量實在太龐大了，IZhna 目前只在草稿中敲定了其中 m 個特殊關卡的難度，這些關卡的難度已經被定死為 1 或 -1 。至於其他沒有被特別定死的關卡，草稿上的預設難度都是 0 。換句話說，草稿可以看作是一個長度為 n 的數列 a ，其中有 m 個元素非零，其餘皆為 0 。

IZhna 將依照以下規則決定所有關卡的最終難度 $c_1, c_2, \dots, c_n$ ：

- 如果草稿中第 i 關的難度 a_i 已經被設定為 1 或 -1 ，則無法更改（最終難度 c_i 必須等於 a_i ）。
- 如果草稿中第 i 關的難度 a_i 為 0 （即未被特別定死的關卡），IZhna 可以自由選擇將最終難度 c_i 設定為 -1 、 0 或 1 。

你的任務是計算出 IZhna 總共有多少種不同的方式來設定最終的難度 $(c_1, c_2, \dots, c_n)$ ，使得這款遊戲的設計能夠完美符合上述的前綴和規則。

由於答案可能非常大，請將結果對 $10^9 + 7$ 取模後輸出。

輸入說明

第一行包含兩個整數 n 與 m — 分別代表總關卡數，以及草稿中已經定死難度的非零關卡數量。

接下來的 m 行，每行包含兩個整數 x_j 與 a_{x_j} — 代表第 x_j 關的難度在草稿中被設定為 a_{x_j} 。保證給定的所有 x_j 皆互不相同。

輸出說明

輸出一個整數 — 代表合法設定難度的方法總數，對 $10^9 + 7$ 取模。

測資限制

- $1 \leq n \leq 10^{15}$
- $0 \leq m \leq 2 \cdot 10^5$
- $1 \leq x_j \leq n$
- $a_{x_j} \in \{-1, 1\}$

範例測資

範例輸入 1

5 2
1 1
4 -1

範例輸入 2

1000 3
1 1
333 1
999 1

範例輸出 1

4

範例輸出 2

834013231

H. Hamburger

問題敘述

蟹老闆的美味蟹堡店擴大營業，現在店內共有 n 名負責煎台的員工。製作一個完美的美味蟹堡需要 t 分鐘。為了確保蟹堡的秘方風味一致，每個蟹堡必須由同一位員工從頭煎到尾，且因為每個蟹堡講求精緻，每位員工一天最多只會負責製作一個蟹堡。

每位員工都有各自的排班時間，第 i 名員工當天的打卡上班時刻為 l_i ，下班時刻為 r_i ，他們只能在這段期間內留在煎台工作。

今天，收銀員章魚哥接到了 q 筆大量的急件外送詢問（每筆詢問互相獨立，互不影響）。第 j 筆詢問表示：如果客人在時刻 ql 點餐，要求在時刻 qr 前（含）拿到 x 個美味蟹堡，店內的排班是否有辦法如期交貨？

請注意，員工必須在客人點餐後（即時刻 ql 起）才能開始煎蟹堡，且多位員工可以同时煎美味蟹堡。

輸入說明

第一行包含三個整數 n, q, t ，分別代表員工數量、詢問次數以及製作單一蟹堡所需的分鐘數。

接下來 n 行，每行包含兩個整數 l_i 與 r_i ，代表第 i 名員工的上班與下班時刻。

接下來 q 行，每行包含三個整數 ql, qr, x ，代表該筆詢問的點餐時刻、要求交貨時刻，以及需要的蟹堡數量。

輸出說明

對於每筆詢問輸出獨立的一行。

如果店內的員工能夠如期完成 x 個美味蟹堡，請輸出 Yes；否則輸出 No。

測資限制

- $1 \leq n, q \leq 10^5$
- $1 \leq t \leq 10^9$
- $1 \leq l_i < r_i \leq 10^9$

- $1 \leq ql < qr \leq 10^9$
- $1 \leq x \leq n$

範例測資

範例輸入 1

4 4 3
1 6
2 5
7 12
4 8
1 5 1
4 8 2
3 10 2
2 4 1

範例輸出 1

Yes
No
Yes
No

I. I like immediate apologies

問題敘述

有些傷口看起來不深，但沒有在第一時間清理與處理，就會慢慢結痂、變硬，最後留下清楚的疤痕。它不一定會痛，但會一直在那裡，提醒你曾經發生過什麼。

人與人之間的關係也是一樣。

當一個人被冒犯、被誤解、或被無意傷害時，那一刻的感受其實是最鮮明的。如果能馬上被看見、被承認、收到一句真誠地「對不起」，情緒有機會被完整地接住，那就像傷口被即時消毒一樣，不會惡化成更深層、永久的問題。

但如果道歉被延遲甚至忽略，情況就不一樣了。情緒不會因為時間就消失，它會被壓下去、被反覆回想、被重新解讀。原本一次小小的摩擦，最後卻可能變成難以忽視的傷。看似癒合了，卻仍會隱隱作痛。

所以我更在意的，是那個「當下」。不是為了責怪誰，而是希望關係裡的傷口，可以在還是新鮮的時候被處理，而不是等它變成疤之後，再試圖把過去縫回來。

這就是為什麼，我最討厭的就是事後道歉。不是因為不願意原諒，而是因為有些東西一旦錯過，就再也回不到當初。

小帥是個討厭事後道歉的人，理所當然的他認為自己做錯事就該立即道歉，但說的容易做的難，他想請你寫一份程式，判斷他是否每次做錯事都有立即道歉。

立即道歉的定義是：在做錯事事件發生後，下一個事件為道歉。

輸入說明

第一行是 1 個正整數 n ，代表接下來有多少事件。

接下來有一長度為 n 的字串 s ， s 中的每個字元 c 都代表了一次事件。

$$c = \begin{cases} W & \text{做錯事} \\ A & \text{道歉} \\ \text{others} & \text{其他事件} \end{cases}$$

輸出說明

輸出 yes 如果小帥都有立即道歉，否則輸出 no

測資限制

1. n 是不超過 100 的正整數
2. $c \in \{A-Z\}$

範例測資

範例輸入 1

8
AAWAWAWA

範例輸入 2

6
ABCWWA

範例輸入 3

5
ALLOW

範例輸出 1

yes

範例輸出 2

no

範例輸出 3

no